CAMBRIDGE INSTITUTE OF TECHNOLOGY

NORTH CAMPUS

(Affiliated to Visvesvaraya Technological University, Belagavi)

Bengaluru – 562 110, Karnataka

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY MANUAL

DATA STRUCTURES LABORATORY

Course Code: 1BCSL306 | Semester: III | Scheme: 2025 (VTU CBCS)

Credits	01
CIE Marks	50
SEE Marks	50
Total Marks	100
Examination Type	Practical
Software Used	Dev-C++ (MinGW GCC)

Prepared by:

Prof. Krishnareddy R
Faculty In-Charge
Dept. of CSE, CIT-NC

Prof.Bandhavya G
Faculty In-Charge
Dept.of CSE, CIT-NC

Approved by:

Dr. Rajshekhar M Patil
Head of the Department
Dept. of CSE, CIT-NC

Institute & Department Profile

Institute Vision

To become a hub for engineering excellence, innovation and societal impact, nurturing engineers to solve complex challenges through research, quality education and a culture of collaboration and integrity.

Institute Mission

To transform students into accomplished engineers through value added education and industry partnerships, equipping them with the necessary knowledge and skills to address real-world problems and contribute to the advancement of society.

(Source: Official website of Cambridge Institute of Technology, North Campus — northcampus.cambridge.edu.in)

Department Vision

Empowering competent, ethical Computer Science and Engineering graduates to drive innovation, entrepreneurship, and computing advancements nationally and globally.

Department Mission

- Mission 1: Provide quality education in Computer Science through a strong foundation in theory, practical skills, and emerging technologies.
- Mission 2: Promote innovation, entrepreneurship, ethical values, and lifelong learning to prepare graduates for professional excellence and leadership.
- Mission 3: Foster industry collaboration, research, and societal engagement to develop globally competent professionals capable of addressing real-world challenges.

Program Outcomes (POs)

Engineering Graduates of the Computer Science and Engineering programme will be able to:

PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7	Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate knowledge of and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

Program Specific Outcomes (PSOs)

PSO1	Apply the knowledge of programming languages, data structures, operating systems, computer networks, and software development principles.
PSO2	Leverage modern computing tools, algorithms, and databases to analyse complex computing problems to develop innovative software solutions.
PSO3	Develop and implement efficient, secure and intelligent software applications by applying appropriate data, machine learning techniques, and emerging computing technologies.

Course Outcomes (COs) — 1BCSL306: Data Structures Laboratory

At the end of the course, the student will be able to:

CO	Course Outcome
CO1	Develop C programs to implement fundamental data structures and their operations for solving computational problems.
CO2	Make use of appropriate data structures to organize, store, and manipulate data for solving real-world problems using C.
CO3	Build a laboratory record consisting of problem statements, algorithm/design, implementation, program outputs, and result with appropriate inferences.

CO – PO – PSO Mapping

The following matrix maps each Course Outcome (CO) of this laboratory to the Program Outcomes (PO1–PO11) and Program Specific Outcomes (PSO1–PSO3), following the standard NBA outcome-based education (OBE) format. Column headers 1–11 correspond to PO1–PO11 and S1–S3 correspond to PSO1–PSO3.

CO	1	2	3	4	5	6	7	8	9	10	11	S1	S2	S3
CO1	3	2	2	1	2	–	–	–	1	1	–	3	2	2
CO2	2	3	3	2	3	1	–	–	1	1	–	3	3	3
CO3	–	–	–	–	1	–	–	2	2	3	1	1	1	1

Correlation levels: 1 = Low, 2 = Medium, 3 = High; a dash (–) indicates no significant correlation.

Mapping Justification

CO1 – Develop C programs to implement fundamental data structures and their operations: maps strongly to PO1 (application of programming fundamentals) and PO5 (use of Dev-C++/GCC as a modern engineering tool), and moderately to PO2 and PO3 through the logical design of each data structure. It aligns strongly with PSO1 (programming languages and data structures) and moderately with PSO3 (implementing efficient software applications).

CO2 – Make use of appropriate data structures to organize, store, and manipulate data for solving real-world problems: maps strongly to PO2, PO3 and PO5, since selecting and adapting the correct structure (stack, queue, tree, graph, hash table) for a problem requires analysis and design; it maps moderately to PO1, PO4 and PO6, and strongly to PSO1, PSO2 and PSO3 as it draws on data structures, algorithmic tools, and efficient application design together.

CO3 – Build a laboratory record with problem statements, algorithm/design, implementation, outputs, and inferences: maps strongly to PO10 (technical communication and documentation) and moderately to PO8, PO9 and PO11 through disciplined, ethical, and independently-managed record-keeping; it maps weakly to PSO1, PSO2 and PSO3 as a reflection of applying the underlying technical skills in a written form.

Table of Contents

Institute & Department Profile (Vision, Mission) **2**
Program Outcomes (POs) **3**
Program Specific Outcomes (PSOs) **3**
Course Outcomes (COs) **4**
CO – PO – PSO Mapping **5**
Course Syllabus — Data Structures Laboratory (1BCSL306) **6**
Suggested Learning Resources **8**
Assessment Structure **9**
General Instructions to Students **10**
Steps to Execute a C Program in Dev-C++ **10**
PART – A: Conventional Experiments **11**
Program A-1: Library Record Management System (Structures & Dynamic Memory) 11
Program A-2: Stack Operations — Push, Pop, Palindrome Check, Overflow/Underflow . 15
Program A-3: Simulation of a Printer Queue using Queue Data Structure 19
Program A-4: Singly Linked List of Student Records 22
Program A-5: Binary Tree Creation (Level Order) and Traversals 26
Program A-6: Graph Representation using Adjacency Matrix (DFS/BFS) 29
PART – B: Open Ended Experiments **32**
Program B-1: Sparse Matrix — 3-Tuple Representation and Matrix Addition 32
Program B-2: Infix to Postfix Expression Conversion Tool using Stack 35
Program B-3: Circular Queue Operations (Floating Point Data) 38
Program B-4: Doubly Linked List of Employee Data (used as Deque) 41
Program B-5: Binary Search Tree (BST) — Creation, Traversal and Search 46
Program B-6: Employee Record Management System using Hashing 49
References **53**

Course Syllabus

DATA STRUCTURES LABORATORY

Course Code	1BCSL306
Scheme	2025
Semester	3
Teaching Hours/Week (L:T:P)	0:0:2
Total Hours of Pedagogy (L:T:P:SL:TW)	0:0:28:2:30
Credits	1
CIE Marks	50
SEE Marks	50
Total Marks	100
Examination Type (SEE)	Practical
Exam Hours	03

Course Outcomes (Course Skill Set)

At the end of the course, the student will be able to:

1. Develop C programs to implement fundamental data structures and their operations for solving computational problems.
2. Make use of appropriate data structures to organize, store, and manipulate data for solving real-world problems using C.
3. Build a laboratory record consisting of problem statements, algorithm/design, implementation, program outputs, and result with appropriate inferences.

Note

- The laboratory syllabus consists of PART-A and PART-B. PART-A has 6 conventional experiments; PART-B has 6 typical open-ended experiments. The maximum marks for the laboratory course are 100.
- Both PART-A and PART-B are considered for CIE and SEE.
- Students have to answer 1 (one) question from PART-A and 1 (one) question from PART-B.
- The question set for SEE shall be from among the experiments under PART-A, evaluated for 70 marks out of the maximum 100 marks.
- The open-ended question set for SEE shall be any other open-ended question, not selected from the experiments under PART-A, evaluated for 30 marks.
- For Continuous Internal Evaluation (CIE) during the semester, classwork shall typically include open-ended questions from Part-B, along with other similar questions designed to enhance the students' skills.

PART – A: Fixed Set of Experiments

1. Develop a Program in C to: (a) define a structure Book with fields Book_ID, Title, Author, Price, Availability Status; (b) dynamically allocate memory to store the details of N books; (c) implement create(), display(), search(), issueBook() and returnBook(); (d) provide a menu-driven interface with Add / Display / Search / Issue / Return / Exit options.
2. Develop a menu-driven Program in C for STACK of Integers (Array Implementation, maximum size MAX): Push, Pop, check Palindrome using Stack, demonstrate Overflow and Underflow, and Display the status of the Stack.
3. Develop a program in C for simulating a Printer Queue using the Queue data structure: add a print job, process (delete) a print job, display all pending print jobs, display the number of pending print jobs, and demonstrate Queue overflow and underflow conditions.
4. Develop a menu-driven Program in C for a Singly Linked List of Student Records: create a linked list using front insertion; display all records and count the number of nodes; insert and delete nodes at the beginning and end of the list; search for a student record using USN.
5. Develop a Program in C to: (a) create a Binary Tree by inserting integer elements entered by the user in level order; (b) implement createTree(), preorder(), inorder() and postorder(); (c) provide a menu-driven interface with the corresponding options.
6. Develop a Program in C for the following operations on a Graph (G) of Cities: (a) create an Adjacency Matrix representing the graph of N cities; (b) print all the nodes reachable from a given starting node in a digraph using the DFS/BFS method.

PART – B: Open Ended Experiments

Open-ended experiments are a type of laboratory activity where the outcome is not predetermined and students are given the freedom to explore, design, and conduct the experiment based on the problem statements as per the concepts defined by the course coordinator. It encourages creativity, critical thinking, and inquiry-based learning.

1. Develop a C program to: (a) read two sparse matrices of the same order; (b) convert both into 3-tuple representation; (c) perform matrix addition; (d) display the resultant matrix in both normal matrix form and 3-tuple form.
2. Develop a C-based Expression Conversion tool that uses a stack data structure to convert arithmetic expressions from infix notation to postfix notation. The tool should support alphanumeric operands, parentheses, and the operators (+, -, *, /, %, ^). Analyze the correctness of the conversion by testing various valid expressions.
3. Develop a menu-driven Program in C for the following operations on a Circular Queue (integer/floating point data): insert an element; delete an element; display the contents; demonstrate overflow and underflow conditions.
4. Develop a menu-driven Program in C for the following operations on a Doubly Linked List (DLL) of Employee Data (SSN, Name, Dept, Designation, Sal, PhNo): create a DLL of N employees using end insertion; display the status and count the number of nodes; perform Insertion and Deletion at the End; perform Insertion and Deletion at the Front; demonstrate how this DLL can be used as a Double Ended Queue.
5. Develop a menu-driven Program in C for the following operations on a Binary Search Tree (BST) of Integers: create a BST of N Integers; traverse the BST in Inorder, Preorder and Postorder; search the BST for a given element (KEY) and report the appropriate message.
6. Develop a Program in C for an Employee Record Management System using Hashing: create a hash table to store employee records using Employee ID as the key; insert employee records using the hash function H: K → L as H(K) = K mod m; resolve collisions using Linear Probing; search for an employee record

using a given Employee ID; display the contents of the hash table along with the corresponding memory locations.

Suggested Learning Resources

Text Books

- Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, “Fundamentals of Data Structures in C”, 2nd Edition, Universities Press, 2023.

Reference Books / Manuals

- Reema Thareja, “Data Structures using C”, 3rd Edition, Oxford Press, 2023.
- A. M. Tenenbaum, “Data Structures using C”, PHI, 1989.
- Seymour Lipschutz, “Data Structures”, Schaum's Outlines, Revised 1st Edition, McGraw Hill, 2014.

Web Links and e-Resources

- GNU-GCC Documentation: https://gcc.gnu.org/
- Virtual Labs on Data Structures-1: https://ds1-iiith.vlabs.ac.in/List%20of%20experiments.html
- DSA Tutorial (GeeksforGeeks): https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/
- NPTEL Course on DSA: https://nptel.ac.in/courses/106106130
- NPTEL Course on Introduction to C Programming: https://nptel.ac.in/courses/106104128

Assessment Structure

The assessment for each course is equally divided between Continuous Internal Evaluation (CIE) and the Semester End Examination (SEE), with each component carrying 50% weight age (i.e., 50 marks each).

The CIE marks awarded shall be based on the continuous evaluation of the laboratory report using a defined set of rubrics. Each experiment report can be evaluated for 30 marks; the total marks for all reports shall be scaled down to 30 marks. The laboratory test (duration 03 hours), conducted at the end of the last week of the semester or after completion of all the experiments (whichever is earlier), shall be conducted for 50 marks and scaled down to 20 marks. The final average of marks for report and test shall be out of 50 marks. For both CIE and SEE, the student is required to conduct one experiment each from both Part A and Part B.

- To qualify and become eligible to appear for SEE, in the CIE component, a student must secure a minimum of 40% of 50 marks, i.e., 20 marks.
- To pass the SEE component, a student must secure a minimum of 35% of 50 marks, i.e., 18 marks.
- A student is deemed to have successfully completed the course if the combined total of CIE and SEE is at least 40 out of 100 marks.

General Instructions to Students

- Students must bring the laboratory record, observation notebook and the syllabus copy to every laboratory session.
- Study the problem statement carefully, design the algorithm and trace it by hand for a small input before writing the C program.
- Open Dev-C++, create a New Source File, save it with a meaningful name and the extension .c before compiling.
- Compile the program using Execute → Compile (Ctrl+F9); resolve all compiler errors and warnings before execution.
- Run the program using Execute → Run (Ctrl+F10) and test it thoroughly with normal, boundary and invalid inputs.
- Record the aim, algorithm, program and sample output of every experiment neatly in the laboratory record.
- Do not copy programs from others; understand the logic thoroughly, as viva-voce questions will be asked on every experiment.
- Maintain discipline, and switch off the systems properly at the end of every laboratory session.

Steps to Execute a C Program in Dev-C++

1. Open Dev-C++ from the Start Menu / Desktop shortcut.
2. Click File → New → Source File (or press Ctrl+N) to open a new blank editor.
3. Type or paste the C program into the editor window.
4. Click File → Save As, and save the file with a ".c" extension, e.g., program1.c.
5. Click Execute → Compile (or press Ctrl+F9) to compile the program. Fix any errors shown in the Compiler Log tab.
6. Click Execute → Run (or press Ctrl+F10) to run the compiled program; alternatively use Execute → Compile & Run (F11) to do both in one step.
7. Enter the required inputs in the console window that opens, and observe the output.
8. Press any key to close the console window after program execution completes.

PART – A: Conventional Experiments

The following six experiments constitute Part A of the Data Structures Laboratory syllabus. Each program has been compiled and executed using Dev-C++ (MinGW GCC) and verified to run without errors, with sample outputs recorded below.

Program A-1 Library Record Management System using Structures and Dynamic Memory Allocation

Course Outcomes Mapped: CO1, CO2

Aim:

To develop a menu-driven C program to create, display, search, issue and return books in a library using a dynamically allocated array of structures.

Algorithm:

1. Start.
2. Define a structure Book with members Book_ID, Title, Author, Price and Status.
3. Display the menu and read the user's choice.
4. If choice = 1 (Add Book Records): read N, dynamically allocate memory for N structures using malloc(), read the details of each book and initialise Status to "Available".
5. If choice = 2 (Display All Book Records): traverse the allocated array and print the details of every book; if no records exist, display an appropriate message.
6. If choice = 3 (Search Book by Book ID): read a Book ID, perform a linear search through the array and display the record if found, else display "not found".
7. If choice = 4 (Issue a Book): search for the Book ID; if found and the Status is "Available", change it to "Issued"; otherwise display an appropriate message.
8. If choice = 5 (Return a Book): search for the Book ID; if found and the Status is "Issued", change it to "Available"; otherwise display an appropriate message.
9. If choice = 6 (Exit): free the dynamically allocated memory and terminate the program.
10. Repeat steps 3 to 9 until the user selects Exit.
11. Stop.

Program:

```
/* PART A - PROGRAM 1
   Library Management System using dynamically allocated array of structures.
   Operations: create, display, search, issueBook, returnBook (menu driven)
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Book
{
    int Book_ID;
    char Title[50];
    char Author[50];
    float Price;
    char Status[15];   /* Available / Issued */
};
```

```
struct Book *list;   /* pointer to dynamically allocated array */
int n;               /* number of books */

void create()
{
   int i;
   printf("Enter number of books: ");
   scanf("%d", &n);
   list = (struct Book *) malloc(n * sizeof(struct Book));
   if (list == NULL)
   {
      printf("Memory allocation failed!\n");
      exit(0);
   }
   for (i = 0; i < n; i++)
   {
      printf("\nEnter details of book %d\n", i + 1);
      printf("Book ID   : ");
      scanf("%d", &list[i].Book_ID);
      printf("Title     : ");
      scanf(" %[^\n]", list[i].Title);
      printf("Author    : ");
      scanf(" %[^\n]", list[i].Author);
      printf("Price     : ");
      scanf("%f", &list[i].Price);
      strcpy(list[i].Status, "Available");
   }
}

void display()
{
   int i;
   if (n == 0)
   {
      printf("\nNo book records available. Please add records first.\n");
      return;
   }
   printf("\n%-8s%-25s%-20s%-10s%-10s\n", "ID", "Title", "Author", "Price", "Status");
   printf("-------------------------------------------------------------------\n");
   for (i = 0; i < n; i++)
      printf("%-8d%-25s%-20s%-10.2f%-10s\n", list[i].Book_ID, list[i].Title,
            list[i].Author, list[i].Price, list[i].Status);
}

int search(int id)
{
   int i;
   for (i = 0; i < n; i++)
      if (list[i].Book_ID == id)
         return i;
   return -1;
}

void searchBook()
```

```
{
    int id, pos;
    printf("Enter Book ID to search: ");
    scanf("%d", &id);
    pos = search(id);
    if (pos == -1)
        printf("Book with ID %d not found.\n", id);
    else
    {
        printf("\nBook Found:\n");
        printf("ID: %d, Title: %s, Author: %s, Price: %.2f, Status: %s\n",
               list[pos].Book_ID, list[pos].Title, list[pos].Author,
               list[pos].Price, list[pos].Status);
    }
}

void issueBook()
{
    int id, pos;
    printf("Enter Book ID to issue: ");
    scanf("%d", &id);
    pos = search(id);
    if (pos == -1)
        printf("Book not found.\n");
    else if (strcmp(list[pos].Status, "Issued") == 0)
        printf("Book is already issued.\n");
    else
    {
        strcpy(list[pos].Status, "Issued");
        printf("Book \"%s\" issued successfully.\n", list[pos].Title);
    }
}

void returnBook()
{
    int id, pos;
    printf("Enter Book ID to return: ");
    scanf("%d", &id);
    pos = search(id);
    if (pos == -1)
        printf("Book not found.\n");
    else if (strcmp(list[pos].Status, "Available") == 0)
        printf("Book was not issued.\n");
    else
    {
        strcpy(list[pos].Status, "Available");
        printf("Book \"%s\" returned successfully.\n", list[pos].Title);
    }
}

int main()
{
    int choice;
    n = 0;
```

```
    list = NULL;
    do
    {
        printf("\n----- LIBRARY MANAGEMENT SYSTEM -----\n");
        printf("1. Add Book Records\n");
        printf("2. Display All Book Records\n");
        printf("3. Search Book by Book ID\n");
        printf("4. Issue a Book\n");
        printf("5. Return a Book\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: create(); break;
            case 2: display(); break;
            case 3: searchBook(); break;
            case 4: issueBook(); break;
            case 5: returnBook(); break;
            case 6: printf("Exiting...\n"); break;
            default: printf("Invalid choice! Try again.\n");
        }
    } while (choice != 6);

    free(list);
    return 0;
}
```

Sample Output:

```
----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice: Enter number of books:
Enter details of book 1
Book ID   : Title     : Author    : Price     :
Enter details of book 2
Book ID   : Title     : Author    : Price     :
Enter details of book 3
Book ID   : Title     : Author    : Price     :
----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice:
```

```
ID      Title                 Author            Price     Status
-------------------------------------------------------------------------
101     Data Structures       Reema Thareja      650.00    Available
102     Java Complete Reference  Herbert Schildt    750.00    Available
103     Operating Systems     Silberschatz       600.00    Available

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice: Enter Book ID to search:
Book Found:
ID: 102, Title: Java Complete Reference, Author: Herbert Schildt, Price: 750.00, Status: Available

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice: Enter Book ID to issue: Book "Java Complete Reference" issued successfully.

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice:
ID      Title                          Author            Price         Status
-------------------------------------------------------------------------
101     Data Structures       Reema Thareja       650.00    Available
102     Java Complete Reference  Herbert Schildt    750.00      Issued
103     Operating Systems     Silberschatz        600.00    Available

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice: Enter Book ID to return: Book "Java Complete Reference" returned successfully.

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
```

```
5. Return a Book
6. Exit
Enter your choice:
ID      Title                 Author            Price     Status
---------------------------------------------------------------------
101     Data Structures       Reema Thareja     650.00    Available
102     Java Complete Reference  Herbert Schildt   750.00    Available
103     Operating Systems     Silberschatz      600.00    Available

----- LIBRARY MANAGEMENT SYSTEM -----
1. Add Book Records
2. Display All Book Records
3. Search Book by Book ID
4. Issue a Book
5. Return a Book
6. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- What is dynamic memory allocation? Which library function is used for it in C?
- Differentiate between a structure and a union.
- What is the time complexity of linear search?
- Why is malloc() preferred over declaring a fixed size array here?
- What happens if malloc() fails to allocate memory?

Program A-2 Stack Operations (Array Implementation) — Push, Pop, Palindrome Check, Overflow/Underflow

Course Outcomes Mapped: CO1, CO2

Aim:

To implement Stack operations (Push, Pop, Display) using arrays, demonstrate Stack Overflow and Underflow conditions, and use a Stack to check whether a given sequence of integers is a palindrome.

Algorithm:

1. Start.
2. Declare an integer array stack[MAX] and initialise top = -1.
3. push(x): if top == MAX-1, display "Stack Overflow"; else increment top and store x at stack[top].
4. pop(): if top == -1, display "Stack Underflow"; else return stack[top] and decrement top.
5. displayStatus(): if the stack is not empty, print elements from stack[top] down to stack[0].
6. checkPalindrome(): read a sequence of N integers into an array and also push them onto a local stack; compare arr[i] with the i-th element from the top of the stack for all i; if every pair matches, the sequence is a palindrome, else it is not.
7. demoOverflowUnderflow(): push (MAX+2) elements to demonstrate overflow, then pop (MAX+2) elements to demonstrate underflow.
8. Repeat according to the user's menu choice until Exit is selected.
9. Stop.

Program:

```
/* PART A - PROGRAM 2
   Stack of Integers using Array (max size MAX)
   Operations: Push, Pop, Palindrome check, Overflow/Underflow demo, Display status
*/
#include <stdio.h>
#define MAX 10

int stack[MAX];
int top = -1;

void push(int x)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow! Cannot push %d.\n", x);
        return;
    }
    stack[++top] = x;
    printf("%d pushed onto stack.\n", x);
}

int pop()
{
    if (top == -1)
    {
        printf("Stack Underflow! Cannot pop.\n");
        return -9999;
    }
    return stack[top--];
```

```
}

void displayStatus()
{
    int i;
    if (top == -1)
    {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack contents (top to bottom): ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

void checkPalindrome()
{
    int arr[MAX], count = 0, i, isPalin = 1;
    printf("Enter number of elements to check for palindrome (<=%d): ", MAX);
    scanf("%d", &count);
    if (count > MAX)
    {
        printf("Too many elements.\n");
        return;
    }
    printf("Enter %d integers: ", count);
    for (i = 0; i < count; i++)
        scanf("%d", &arr[i]);

    /* push all elements onto a fresh local stack simulation */
    int localStack[MAX], localTop = -1;
    for (i = 0; i < count; i++)
        localStack[++localTop] = arr[i];

    for (i = 0; i < count; i++)
    {
        if (arr[i] != localStack[localTop - i])
        {
            isPalin = 0;
            break;
        }
    }
    if (isPalin)
        printf("The sequence IS a palindrome.\n");
    else
        printf("The sequence is NOT a palindrome.\n");
}

void demoOverflowUnderflow()
{
    int i;
    printf("Demonstrating Overflow: pushing %d elements into a stack of size %d\n", MAX + 2, MAX);
    top = -1;
```

```
    for (i = 1; i <= MAX + 2; i++)
        push(i);

    printf("\nDemonstrating Underflow: popping %d elements from stack that has %d elements\n", MAX
+ 2, top + 1);
    for (i = 1; i <= MAX + 2; i++)
    {
        int val = pop();
        if (val != -9999)
            printf("Popped: %d\n", val);
    }
}

int main()
{
    int choice, val;
    do
    {
        printf("\n----- STACK OPERATIONS (Array Implementation) -----\n");
        printf("1. Push an Element onto Stack\n");
        printf("2. Pop an Element from Stack\n");
        printf("3. Check Palindrome using Stack\n");
        printf("4. Demonstrate Overflow and Underflow\n");
        printf("5. Display Status of Stack\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &val);
                push(val);
                break;
            case 2:
                val = pop();
                if (val != -9999)
                    printf("Popped element: %d\n", val);
                break;
            case 3: checkPalindrome(); break;
            case 4: demoOverflowUnderflow(); break;
            case 5: displayStatus(); break;
            case 6: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 6);
    return 0;
}
```

Sample Output:

```
----- STACK OPERATIONS (Array Implementation) -----
```

1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Enter value to push: 10 pushed onto stack.

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Enter value to push: 20 pushed onto stack.

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Enter value to push: 30 pushed onto stack.

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Stack contents (top to bottom): 30 20 10

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Enter number of elements to check for palindrome (<=10): Enter 5 integers: The sequence IS a palindrome.

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Enter number of elements to check for palindrome (<=10): Enter 4 integers: The sequence is NOT a palindrome.

```
----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Demonstrating Overflow: pushing 12 elements into a stack of size 10
1 pushed onto stack.
2 pushed onto stack.
3 pushed onto stack.
4 pushed onto stack.
5 pushed onto stack.
6 pushed onto stack.
7 pushed onto stack.
8 pushed onto stack.
9 pushed onto stack.
10 pushed onto stack.
Stack Overflow! Cannot push 11.
Stack Overflow! Cannot push 12.

Demonstrating Underflow: popping 12 elements from stack that has 10 elements
Popped: 10
Popped: 9
Popped: 8
Popped: 7
Popped: 6
Popped: 5
Popped: 4
Popped: 3
Popped: 2
Popped: 1
Stack Underflow! Cannot pop.
Stack Underflow! Cannot pop.

----- STACK OPERATIONS (Array Implementation) -----
1. Push an Element onto Stack
2. Pop an Element from Stack
3. Check Palindrome using Stack
4. Demonstrate Overflow and Underflow
5. Display Status of Stack
6. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- What is the LIFO principle? Give a real-world example.
- What are the applications of a stack?
- What is Stack Overflow and Stack Underflow? How are they detected?
- What is the time complexity of push and pop operations?
- Give two different approaches to check whether a sequence is a palindrome.

Program A-3 Simulation of a Printer Queue using Queue Data Structure

Course Outcomes Mapped: CO1, CO2

Aim:

To simulate a Printer Queue using a linear Queue (array implementation) supporting addition of print jobs, processing (deletion) of jobs, and demonstration of overflow and underflow conditions.

Algorithm:

1. Start.
2. Declare an array queue[MAX] of job names and initialise front = rear = -1.
3. addJob(job): if rear == MAX-1, display "Queue Overflow"; else if front == -1, set front = 0; increment rear and store the job at queue[rear].
4. process Job(): if the queue is empty (front == -1 or front > rear), display "Queue Underflow"; else display and remove the job at queue[front], then increment front.
5. display Jobs(): traverse from front to rear and print all pending jobs.
6. count Jobs(): print the number of jobs currently waiting as (rear - front + 1).
7. demo Overflow Underflow(): add (MAX+2) jobs to demonstrate overflow, then process (MAX+2) jobs to demonstrate underflow.
8. Repeat according to the user's menu choice until Exit is selected.
9. Stop.

Program:

```
/* PART A - PROGRAM 3
   Simulate a Printer Queue using linear Queue (array implementation)
   Operations: Add print job, Process(delete) job, Display pending jobs,
               Display count, Demonstrate overflow/underflow
*/
#include <stdio.h>
#include <string.h>
#define MAX 5

char queue[MAX][30];
int front = -1, rear = -1;

void addJob(char job[])
{
    if (rear == MAX - 1)
    {
        printf("Queue Overflow! Cannot add job \"%s\". Printer queue is full.\n", job);
        return;
    }
    if (front == -1)
        front = 0;
    rear++;
    strcpy(queue[rear], job);
    printf("Print job \"%s\" added to queue.\n", job);
}

void processJob()
{
    if (front == -1 || front > rear)
    {
```

```
        printf("Queue Underflow! No print jobs to process.\n");
        front = rear = -1;
        return;
    }
    printf("Processing (removing) print job: \"%s\"\n", queue[front]);
    front++;
    if (front > rear)
        front = rear = -1;
}

void displayJobs()
{
    int i;
    if (front == -1 || front > rear)
    {
        printf("No pending print jobs.\n");
        return;
    }
    printf("Pending print jobs: ");
    for (i = front; i <= rear; i++)
        printf("[%s] ", queue[i]);
    printf("\n");
}

void countJobs()
{
    if (front == -1 || front > rear)
        printf("Number of print jobs waiting: 0\n");
    else
        printf("Number of print jobs waiting: %d\n", rear - front + 1);
}

void demoOverflowUnderflow()
{
    int i;
    char job[30];
    front = rear = -1;
    printf("Demonstrating Overflow: adding %d jobs to a queue of size %d\n", MAX + 2, MAX);
    for (i = 1; i <= MAX + 2; i++)
    {
        sprintf(job, "Job%d", i);
        addJob(job);
    }
    printf("\nDemonstrating Underflow: processing %d jobs from queue with %d jobs\n",
           MAX + 2, (front == -1) ? 0 : rear - front + 1);
    for (i = 1; i <= MAX + 2; i++)
        processJob();
}

int main()
{
    int choice;
    char job[30];
    do
```

```
    {
        printf("\n----- PRINTER QUEUE SIMULATION -----\n");
        printf("1. Add a print job to the Printer Queue\n");
        printf("2. Process (delete) a print job\n");
        printf("3. Display all pending print jobs\n");
        printf("4. Display number of print jobs waiting\n");
        printf("5. Demonstrate Overflow and Underflow\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter job name: ");
                scanf("%s", job);
                addJob(job);
                break;
            case 2: processJob(); break;
            case 3: displayJobs(); break;
            case 4: countJobs(); break;
            case 5: demoOverflowUnderflow(); break;
            case 6: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 6);
    return 0;
}
```

Sample Output:

```
----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Enter job name: Print job "Job1" added to queue.

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Enter job name: Print job "Job2" added to queue.

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
```

4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Enter job name: Print job "Job3" added to queue.

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Pending print jobs: [Job1] [Job2] [Job3]

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Number of print jobs waiting: 3

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Processing (removing) print job: "Job1"

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Pending print jobs: [Job2] [Job3]

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Number of print jobs waiting: 2

----- PRINTER QUEUE SIMULATION -----
1. Add a print job to the Printer Queue
2. Process (delete) a print job
3. Display all pending print jobs

```
4. Display number of print jobs waiting
5. Demonstrate Overflow and Underflow
6. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- What is the FIFO principle? Give a real-world example.
- Differentiate between a linear queue and a circular queue.
- What is the drawback of a simple (linear) queue implemented using an array?
- List real-world applications of the Queue data structure.
- What is the time complexity of enqueue and dequeue operations?

Program A-4 Singly Linked List of Student Records

Course Outcomes Mapped: CO1, CO2

Aim:

To implement a Singly Linked List of Student records (USN, Name) that is built using front insertion, and supports display with node count, insertion/deletion at the beginning and end, and searching by USN.

Algorithm:

1. Start.
2. Define a self-referential structure Node with members USN, Name and a pointer next.
3. createFront(): for each of the N students, dynamically allocate a new node, read its data, set newNode->next = head, and then set head = newNode (front insertion).
4. displayAndCount(): starting at head, traverse the list printing each node's data and incrementing a counter; print the final count.
5. insertBeginning(): allocate a new node, set its next to the current head, and update head to the new node.
6. insertEnd(): allocate a new node, traverse to the last node, and link it as the next of the last node.
7. deleteBeginning(): store head in a temporary pointer, move head to head->next, and free the temporary node.
8. deleteEnd(): traverse to the second-last node, free the last node, and set the second-last node's next to NULL.
9. searchUSN(): traverse the list comparing each node's USN with the key using strcmp(); report the position if found.
10. Repeat according to the user's menu choice until Exit is selected.
11. Stop.

Program:

```
/* PART A - PROGRAM 4
   Singly Linked List of Student Records
   Operations: Create using front insertion, Display + count, Insert/Delete
               at beginning and end, Search by USN
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node
{
    char USN[15];
    char Name[30];
    struct Node *next;
};

struct Node *head = NULL;

/* Create list using FRONT insertion */
void createFront()
{
    int n, i;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
```

```
    {
        struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
        printf("Enter USN for student %d: ", i + 1);
        scanf("%s", newNode->USN);
        printf("Enter Name for student %d: ", i + 1);
        scanf("%s", newNode->Name);
        newNode->next = head;
        head = newNode;
    }
    printf("Linked list created using front insertion.\n");
}

void displayAndCount()
{
    struct Node *p = head;
    int count = 0;
    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }
    printf("\n%-15s%-20s\n", "USN", "Name");
    printf("--------------------------------\n");
    while (p != NULL)
    {
        printf("%-15s%-20s\n", p->USN, p->Name);
        count++;
        p = p->next;
    }
    printf("Total number of nodes: %d\n", count);
}

void insertBeginning()
{
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter USN: ");
    scanf("%s", newNode->USN);
    printf("Enter Name: ");
    scanf("%s", newNode->Name);
    newNode->next = head;
    head = newNode;
    printf("Node inserted at beginning.\n");
}

void insertEnd()
{
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    printf("Enter USN: ");
    scanf("%s", newNode->USN);
    printf("Enter Name: ");
    scanf("%s", newNode->Name);
    newNode->next = NULL;

    if (head == NULL)
```

```
    {
        head = newNode;
        return;
    }
    struct Node *p = head;
    while (p->next != NULL)
        p = p->next;
    p->next = newNode;
    printf("Node inserted at end.\n");
}

void deleteBeginning()
{
    if (head == NULL)
    {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted node with USN: %s\n", temp->USN);
    free(temp);
}

void deleteEnd()
{
    if (head == NULL)
    {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
    if (head->next == NULL)
    {
        printf("Deleted node with USN: %s\n", head->USN);
        free(head);
        head = NULL;
        return;
    }
    struct Node *p = head;
    while (p->next->next != NULL)
        p = p->next;
    printf("Deleted node with USN: %s\n", p->next->USN);
    free(p->next);
    p->next = NULL;
}

void searchUSN()
{
    char key[15];
    int pos = 1;
    printf("Enter USN to search: ");
    scanf("%s", key);
    struct Node *p = head;
    while (p != NULL)
```

```
    {
        if (strcmp(p->USN, key) == 0)
        {
            printf("Record found at position %d -> USN: %s, Name: %s\n", pos, p->USN, p->Name);
            return;
        }
        p = p->next;
        pos++;
    }
    printf("USN %s not found in the list.\n", key);
}

int main()
{
    int choice, sub;
    do
    {
        printf("\n----- SINGLY LINKED LIST OF STUDENT RECORDS -----\n");
        printf("1. Create linked list using front insertion\n");
        printf("2. Display all records and count nodes\n");
        printf("3. Insert / Delete at beginning or end\n");
        printf("4. Search for a student record using USN\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: createFront(); break;
            case 2: displayAndCount(); break;
            case 3:
                printf("1.Insert Beginning 2.Insert End 3.Delete Beginning 4.Delete End\nChoice: ");
                scanf("%d", &sub);
                if (sub == 1) insertBeginning();
                else if (sub == 2) insertEnd();
                else if (sub == 3) deleteBeginning();
                else if (sub == 4) deleteEnd();
                else printf("Invalid sub-choice.\n");
                break;
            case 4: searchUSN(); break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 5);
    return 0;
}
```

Sample Output:

```
----- SINGLY LINKED LIST OF STUDENT RECORDS -----
1. Create linked list using front insertion
2. Display all records and count nodes
3. Insert / Delete at beginning or end
```

4. Search for a student record using USN
5. Exit
Enter your choice: Enter number of students: Enter USN for student 1: Enter Name for student 1: Enter USN for student 2: Enter Name for student 2: Enter USN for student 3: Enter Name for student 3: Linked list created using front insertion.

----- SINGLY LINKED LIST OF STUDENT RECORDS -----
1. Create linked list using front insertion
2. Display all records and count nodes
3. Insert / Delete at beginning or end
4. Search for a student record using USN
5. Exit
Enter your choice: Enter USN to search: Record found at position 2 -> USN: 1BM21CS002, Name: Bob

----- SINGLY LINKED LIST OF STUDENT RECORDS -----
1. Create linked list using front insertion
2. Display all records and count nodes
3. Insert / Delete at beginning or end
4. Search for a student record using USN
5. Exit
Enter your choice: Enter USN to search: USN 1BM21CS999 not found in the list.

----- SINGLY LINKED LIST OF STUDENT RECORDS -----
1. Create linked list using front insertion
2. Display all records and count nodes
3. Insert / Delete at beginning or end
4. Search for a student record using USN
5. Exit
Enter your choice: Exiting...

Viva-Voce Questions:

- What is a linked list? How does it differ from an array?
- Why is front insertion O(1) while end insertion is O(n) in a singly linked list?
- What happens to the pointer of the last node in a singly linked list?
- What is a dangling pointer? How is it avoided while deleting a node?
- What is the advantage of a linked list over an array for insertion and deletion?

Program A-5 Binary Tree Creation (Level Order) and Preorder, Inorder, Postorder Traversals

Course Outcomes Mapped: CO1, CO2

Aim:

To create a Binary Tree by inserting integer elements entered by the user in level order using dynamic memory allocation, and to display the tree using Preorder, Inorder and Postorder traversals.

Algorithm:

1. Start.
2. Define a structure Node with members data, left and right.
3. createTree(): maintain an auxiliary queue of tree-node pointers; for the first value create the root and enqueue it; for every subsequent value, examine the node at the front of the auxiliary queue — if its left child is empty, attach the new node as its left child, else attach it as its right child and dequeue the front node; enqueue every newly created node.
4. preorder(t): if t is not NULL, visit t, then recursively traverse the left subtree, then the right subtree.
5. inorder(t): if t is not NULL, recursively traverse the left subtree, visit t, then recursively traverse the right subtree.
6. postorder(t): if t is not NULL, recursively traverse the left subtree, then the right subtree, then visit t.
7. Repeat according to the user's menu choice until Exit is selected.
8. Stop.

Program:

```
/* PART A - PROGRAM 5
   Binary Tree created by inserting elements in LEVEL ORDER (as a complete
   binary tree, filled left to right) using dynamic memory allocation.
   Operations: createTree, preorder, inorder, postorder (menu driven)
*/
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *left, *right;
};

struct Node *root = NULL;

/* Queue used only to build the tree in level order */
struct Node *queueArr[100];
int qFront = 0, qRear = -1;

void enqueue(struct Node *n)
{
    queueArr[++qRear] = n;
}
struct Node *dequeue()
{
    return queueArr[qFront++];
}
int isQueueEmpty()
{
```

```
    return qFront > qRear;
}

struct Node *createNode(int val)
{
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

void createTree()
{
    int n, i, val;
    root = NULL;
    qFront = 0; qRear = -1;
    printf("Enter number of elements to insert into the Binary Tree: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &val);
        struct Node *newNode = createNode(val);

        if (root == NULL)
        {
            root = newNode;
            enqueue(root);
            continue;
        }

        struct Node *front = queueArr[qFront];
        if (front->left == NULL)
        {
            front->left = newNode;
        }
        else if (front->right == NULL)
        {
            front->right = newNode;
            qFront++;   /* this node is now full, move to next */
        }
        enqueue(newNode);
    }
    printf("Binary tree created successfully in level order.\n");
}

void preorder(struct Node *t)
{
    if (t != NULL)
    {
        printf("%d ", t->data);
        preorder(t->left);
        preorder(t->right);
```

```
    }
}

void inorder(struct Node *t)
{
    if (t != NULL)
    {
        inorder(t->left);
        printf("%d ", t->data);
        inorder(t->right);
    }
}

void postorder(struct Node *t)
{
    if (t != NULL)
    {
        postorder(t->left);
        postorder(t->right);
        printf("%d ", t->data);
    }
}

int main()
{
    int choice;
    do
    {
        printf("\n----- BINARY TREE (Level Order Creation) -----\n");
        printf("1. Create Binary Tree\n");
        printf("2. Display Preorder Traversal\n");
        printf("3. Display Inorder Traversal\n");
        printf("4. Display Postorder Traversal\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: createTree(); break;
            case 2:
                if (root == NULL) { printf("Tree is empty!\n"); break; }
                printf("Preorder  : "); preorder(root); printf("\n");
                break;
            case 3:
                if (root == NULL) { printf("Tree is empty!\n"); break; }
                printf("Inorder   : "); inorder(root); printf("\n");
                break;
            case 4:
                if (root == NULL) { printf("Tree is empty!\n"); break; }
                printf("Postorder : "); postorder(root); printf("\n");
                break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
```

```
        }
    } while (choice != 5);
    return 0;
}
```

Sample Output:

```
----- BINARY TREE (Level Order Creation) -----
1. Create Binary Tree
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit
Enter your choice: Enter number of elements to insert into the Binary Tree: Enter element 1: Enter element 2: Enter element 3: Enter element 4: Enter element 5: Enter element 6: Enter element 7: Binary tree created successfully in level order.

----- BINARY TREE (Level Order Creation) -----
1. Create Binary Tree
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit
Enter your choice: Preorder  : 1 2 4 5 3 6 7

----- BINARY TREE (Level Order Creation) -----
1. Create Binary Tree
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit
Enter your choice: Inorder   : 4 2 5 1 6 3 7

----- BINARY TREE (Level Order Creation) -----
1. Create Binary Tree
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit
Enter your choice: Postorder : 4 5 2 6 7 3 1

----- BINARY TREE (Level Order Creation) -----
1. Create Binary Tree
2. Display Preorder Traversal
3. Display Inorder Traversal
4. Display Postorder Traversal
5. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- Define a binary tree. What is the maximum number of nodes at level k?
- What is the difference between a Binary Tree and a Binary Search Tree?

- What is level order insertion? Why is a queue used to build the tree in level order?
- What are the three standard depth-first traversals of a binary tree?
- Give one application each of preorder, inorder and postorder traversal.

Program A-6 Graph Representation using Adjacency Matrix — Reachability using DFS/BFS

Course Outcomes Mapped: CO1, CO2

Aim:

To represent a graph of N cities using an Adjacency Matrix and to print all the nodes reachable from a given starting node using Depth First Search (DFS) and Breadth First Search (BFS).

Algorithm:

1. Start.
2. Read N, the number of cities (vertices), and the N x N adjacency matrix adj[][], where adj[i][j] = 1 if an edge exists between city i and city j, else 0.
3. DFS(start): push start onto an explicit stack; while the stack is not empty, pop a vertex v; if v is unvisited, mark it visited and print it, then push all unvisited vertices adjacent to v onto the stack.
4. BFS(start): mark start as visited and insert it into a queue; while the queue is not empty, remove a vertex v, print it, and insert every unvisited vertex adjacent to v into the queue, marking it visited.
5. Repeat according to the user's menu choice until Exit is selected.
6. Stop.

Program:

```
/* PART A - PROGRAM 6
   Graph (G) of N Cities represented using Adjacency Matrix.
   a. Create an Adjacency Matrix representing the graph of N cities
   b. Print all the nodes reachable from a given starting node using DFS/BFS
*/
#include <stdio.h>

#define MAXV 20
int adj[MAXV][MAXV];
int visited[MAXV];
int n;   /* number of cities (vertices) */

void createGraph()
{
   int i, j;
   printf("Enter number of cities (vertices): ");
   scanf("%d", &n);
   printf("Enter adjacency matrix (%d x %d), 1 if edge exists else 0:\n", n, n);
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &adj[i][j]);
}

void displayGraph()
{
   int i, j;
   printf("\nAdjacency Matrix:\n");
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n; j++)
         printf("%d ", adj[i][j]);
      printf("\n");
   }
```

```
}

/* stack based DFS */
void DFS(int start)
{
   int stack[MAXV], top = -1, i, v;
   for (i = 0; i < n; i++) visited[i] = 0;

   stack[++top] = start;
   printf("DFS reachable nodes from city %d: ", start);
   while (top != -1)
   {
      v = stack[top--];
      if (!visited[v])
      {
         visited[v] = 1;
         printf("%d ", v);
         /* push neighbours in reverse order so smaller index visited first */
         for (i = n - 1; i >= 0; i--)
            if (adj[v][i] == 1 && !visited[i])
               stack[++top] = i;
      }
   }
   printf("\n");
}

void BFS(int start)
{
   int queue[MAXV], front = 0, rear = -1, i, v;
   for (i = 0; i < n; i++) visited[i] = 0;

   visited[start] = 1;
   queue[++rear] = start;
   printf("BFS reachable nodes from city %d: ", start);
   while (front <= rear)
   {
      v = queue[front++];
      printf("%d ", v);
      for (i = 0; i < n; i++)
         if (adj[v][i] == 1 && !visited[i])
         {
            visited[i] = 1;
            queue[++rear] = i;
         }
   }
   printf("\n");
}

int main()
{
   int choice, start;
   do
   {
      printf("\n----- GRAPH OF CITIES (ADJACENCY MATRIX) -----\n");
```

```
        printf("1. Create Adjacency Matrix of Graph\n");
        printf("2. Display Adjacency Matrix\n");
        printf("3. Find reachable nodes using DFS\n");
        printf("4. Find reachable nodes using BFS\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: createGraph(); break;
            case 2: displayGraph(); break;
            case 3:
                printf("Enter starting city (0 to %d): ", n - 1);
                scanf("%d", &start);
                DFS(start);
                break;
            case 4:
                printf("Enter starting city (0 to %d): ", n - 1);
                scanf("%d", &start);
                BFS(start);
                break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 5);
    return 0;
}
```

Sample Output:

```
----- GRAPH OF CITIES (ADJACENCY MATRIX) -----
1. Create Adjacency Matrix of Graph
2. Display Adjacency Matrix
3. Find reachable nodes using DFS
4. Find reachable nodes using BFS
5. Exit
Enter your choice: Enter number of cities (vertices): Enter adjacency matrix (5 x 5), 1 if edge exists else
0:

----- GRAPH OF CITIES (ADJACENCY MATRIX) -----
1. Create Adjacency Matrix of Graph
2. Display Adjacency Matrix
3. Find reachable nodes using DFS
4. Find reachable nodes using BFS
5. Exit
Enter your choice:
Adjacency Matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
```

```
----- GRAPH OF CITIES (ADJACENCY MATRIX) -----
1. Create Adjacency Matrix of Graph
2. Display Adjacency Matrix
3. Find reachable nodes using DFS
4. Find reachable nodes using BFS
5. Exit
Enter your choice: Enter starting city (0 to 4): DFS reachable nodes from city 0: 0 1 3 4 2

----- GRAPH OF CITIES (ADJACENCY MATRIX) -----
1. Create Adjacency Matrix of Graph
2. Display Adjacency Matrix
3. Find reachable nodes using DFS
4. Find reachable nodes using BFS
5. Exit
Enter your choice: Enter starting city (0 to 4): BFS reachable nodes from city 0: 0 1 2 3 4

----- GRAPH OF CITIES (ADJACENCY MATRIX) -----
1. Create Adjacency Matrix of Graph
2. Display Adjacency Matrix
3. Find reachable nodes using DFS
4. Find reachable nodes using BFS
5. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- What is a graph? Differentiate between directed and undirected graphs.
- Compare the Adjacency Matrix and Adjacency List representations of a graph.
- What is the space complexity of the Adjacency Matrix representation?
- Differentiate between DFS and BFS in terms of the data structure used and the order of traversal.
- List one real-world application each of DFS and BFS.

PART – B: Open Ended Experiments

The following six open-ended experiments constitute Part B of the Data Structures Laboratory syllabus. Students are encouraged to explore variations of these problems and test additional cases beyond those shown here.

Program B-1 Sparse Matrix — 3-Tuple Representation and Matrix Addition

Course Outcomes Mapped: CO1, CO2

Aim:

To read two sparse matrices of the same order, convert both into their 3-tuple (Row, Column, Value) representation, perform matrix addition, and display the resultant matrix in both normal and 3-tuple form.

Algorithm:

1. Start.
2. Read the order (rows, cols) of the sparse matrices, and read the elements of matrices A and B.
3. toTuple(): scan the matrix row-wise; for every non-zero element store its row index, column index and value as one tuple; store (rows, cols, number_of_nonzero_terms) as the 0th tuple.
4. Compute the sum matrix C such that C[i][j] = A[i][j] + B[i][j] for every i, j.
5. Convert C into its 3-tuple representation using the same procedure.
6. Display A, B and C in normal matrix form, followed by their respective 3-tuple representations.
7. Stop.

Program:

```
/* PART B - PROGRAM 1
   a. Read two sparse matrices of the same order
   b. Convert both into 3-tuple representation
   c. Perform matrix addition
   d. Display the resultant matrix in both normal matrix form and 3-tuple form
*/
#include <stdio.h>
#define MAXSIZE 20

void readMatrix(int m[][MAXSIZE], int rows, int cols, const char *name){
    int i, j;
    printf("Enter elements of matrix %s (%d x %d):\n", name, rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &m[i][j]);
}

void toTuple(int m[][MAXSIZE], int rows, int cols, int tuple[][3], int *count)
{
    int i, j, k = 1;
    tuple[0][0] = rows;
    tuple[0][1] = cols;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            if (m[i][j] != 0)
            {
                tuple[k][0] = i;
                tuple[k][1] = j;
```

```
            tuple[k][2] = m[i][j];
            k++;
        }
    tuple[0][2] = k - 1;   /* number of non-zero terms */
    *count = k;
}

void displayMatrix(int m[][MAXSIZE], int rows, int cols)
{
    int i, j;
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }
}

void displayTuple(int tuple[][3], int count)
{
    int i;
    printf(" Row  Col  Value\n");
    for (i = 0; i < count; i++)
        printf("%4d %4d %6d\n", tuple[i][0], tuple[i][1], tuple[i][2]);
}

int main()
{
    int a[MAXSIZE][MAXSIZE], b[MAXSIZE][MAXSIZE], c[MAXSIZE][MAXSIZE];
    int tupleA[MAXSIZE * MAXSIZE][3], tupleB[MAXSIZE * MAXSIZE][3], tupleC[MAXSIZE *
MAXSIZE][3];
    int rows, cols, i, j, countA, countB, countC;

    printf("Enter order of the sparse matrices (rows and columns): ");
    scanf("%d %d", &rows, &cols);

    readMatrix(a, rows, cols, "A");
    readMatrix(b, rows, cols, "B");

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            c[i][j] = a[i][j] + b[i][j];

    toTuple(a, rows, cols, tupleA, &countA);
    toTuple(b, rows, cols, tupleB, &countB);
    toTuple(c, rows, cols, tupleC, &countC);

    printf("\nMatrix A (Normal form):\n");
    displayMatrix(a, rows, cols);
    printf("\nMatrix A (3-Tuple form) [Row Col NonZeroCount = %d %d %d]:\n",
          tupleA[0][0], tupleA[0][1], tupleA[0][2]);
    displayTuple(tupleA + 1, countA - 1);

    printf("\nMatrix B (Normal form):\n");
```

```
    displayMatrix(b, rows, cols);
    printf("\nMatrix B (3-Tuple form) [Row Col NonZeroCount = %d %d %d]:\n",
           tupleB[0][0], tupleB[0][1], tupleB[0][2]);
    displayTuple(tupleB + 1, countB - 1);

    printf("\nResultant Matrix C = A + B (Normal form):\n");
    displayMatrix(c, rows, cols);
    printf("\nResultant Matrix C (3-Tuple form) [Row Col NonZeroCount = %d %d %d]:\n",
           tupleC[0][0], tupleC[0][1], tupleC[0][2]);
    displayTuple(tupleC + 1, countC - 1);

    return 0;
}
```

Sample Output:

```
Enter order of the sparse matrices (rows and columns): Enter elements of matrix A (3 x 3):
Enter elements of matrix B (3 x 3):

Matrix A (Normal form):
  0   0   5
  0   3   0
  6   0   0

Matrix A (3-Tuple form) [Row Col NonZeroCount = 3 3 3]:
 Row  Col  Value
  0    2     5
  1    1     3
  2    0     6

Matrix B (Normal form):
  0   8   0
  0   0   4
  2   0   9

Matrix B (3-Tuple form) [Row Col NonZeroCount = 3 3 4]:
 Row  Col  Value
  0    1     8
  1    2     4
  2    0     2
  2    2     9

Resultant Matrix C = A + B (Normal form):
  0   8   5
  0   3   4
  8   0   9

Resultant Matrix C (3-Tuple form) [Row Col NonZeroCount = 3 3 6]:
 Row  Col  Value
  0    1     8
  0    2     5
  1    1     3
  1    2     4
  2    0     8
```

```
2 2 9
```

Viva-Voce Questions:

- What is a sparse matrix? Give an example.
- Why is the 3-tuple representation more memory-efficient than the normal 2D array representation for a sparse matrix?
- What information is stored in the 0th tuple (header) of the 3-tuple representation?
- What is the time complexity of converting a matrix into its 3-tuple representation?
- List other applications where sparse matrix representation is useful.

Program B-2 Infix to Postfix Expression Conversion Tool using Stack

Course Outcomes Mapped: CO1, CO2

Aim:

To develop a C program using a Stack that converts an arithmetic expression from infix notation to postfix notation, supporting alphanumeric operands, parentheses and the operators +, -, *, /, %, ^.

Algorithm:

1. Start.
2. Initialise an empty character stack to hold operators.
3. Scan the infix expression from left to right, one character at a time.
4. If the current character is an operand (alphanumeric), append it directly to the postfix output string.
5. If the current character is '(', push it onto the stack.
6. If the current character is ')', pop operators from the stack and append them to the output until '(' is encountered; discard the '('.
7. If the current character is an operator, pop and append operators from the stack that have higher precedence, or equal precedence with left associativity, then push the current operator onto the stack.
8. After the entire expression has been scanned, pop all remaining operators from the stack and append them to the output.
9. Display the resulting postfix expression.
10. Stop.

Program:

```
/* PART B - PROGRAM 2
   Expression Conversion Tool: Infix to Postfix using Stack.
   Supports alphanumeric operands, parentheses, and operators + - * / % ^
*/
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100

char stack[MAX];
int top = -1;

void push(char c)
{
    stack[++top] = c;
}
char pop()
{
    return stack[top--];
}
char peek()
{
    return stack[top];
}
int isEmpty()
{
    return top == -1;
}
```

```
int precedence(char op)
{
    if (op == '^') return 3;
    if (op == '*' || op == '/' || op == '%') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int isRightAssociative(char op)
{
    return op == '^';
}

int isOperator(char c)
{
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '%' || c == '^');
}

void infixToPostfix(char infix[], char postfix[])
{
    int i, k = 0;
    top = -1;

    for (i = 0; infix[i] != '\0'; i++)
    {
        char c = infix[i];

        if (isspace(c))
            continue;

        if (isalnum(c))
        {
            postfix[k++] = c;
        }
        else if (c == '(')
        {
            push(c);
        }
        else if (c == ')')
        {
            while (!isEmpty() && peek() != '(')
                postfix[k++] = pop();
            if (!isEmpty())
                pop();   /* remove '(' */
        }
        else if (isOperator(c))
        {
            while (!isEmpty() && peek() != '(' &&
                   (precedence(peek()) > precedence(c) ||
                    (precedence(peek()) == precedence(c) && !isRightAssociative(c))))
            {
                postfix[k++] = pop();
            }
```

```
            push(c);
        }
    }
    while (!isEmpty())
        postfix[k++] = pop();
    postfix[k] = '\0';
}

int main()
{
    char infix[MAX], postfix[MAX];
    char choice;

    do
    {
        printf("\nEnter an infix expression (operands: alphanumeric, operators: + - * / %% ^, and
parentheses):\n");
        scanf(" %[^\n]", infix);

        infixToPostfix(infix, postfix);

        printf("Infix   Expression : %s\n", infix);
        printf("Postfix Expression : %s\n", postfix);

        printf("\nConvert another expression? (y/n): ");
        scanf(" %c", &choice);
    } while (choice == 'y' || choice == 'Y');

    return 0;
}
```

Sample Output:

```
Enter an infix expression (operands: alphanumeric, operators: + - * / % ^, and parentheses):
Infix   Expression : A+B*C-D
Postfix Expression : ABC*+D-

Convert another expression? (y/n):
```

Viva-Voce Questions:

- Why is a stack the natural data structure for infix-to-postfix conversion?
- What is operator precedence and associativity? Why does '^' need special handling?
- What is the advantage of postfix notation over infix notation for expression evaluation?
- How would you evaluate a postfix expression using a stack?
- What is the time complexity of the infix-to-postfix conversion algorithm?

Program B-3 Circular Queue Operations (Integer / Floating Point Data)

Course Outcomes Mapped: CO1, CO2

Aim:

To implement a Circular Queue using an array (floating point data) supporting insertion, deletion, display of contents, and demonstration of overflow and underflow conditions.

Algorithm:

1. Start.
2. Declare an array cq[MAX] and initialise front = rear = -1.
3. insert(x): if (rear+1) % MAX == front, display "Overflow"; else if front == -1, set front = 0; set rear = (rear+1) % MAX and store x at cq[rear].
4. delete(): if front == -1, display "Underflow"; else remove cq[front]; if front == rear, reset front = rear = -1, else set front = (front+1) % MAX.
5. display(): starting at front, print elements circularly up to rear using modulo arithmetic.
6. demoOverflowUnderflow(): insert (MAX+2) elements to demonstrate overflow, then delete (MAX+2) elements to demonstrate underflow.
7. Repeat according to the user's menu choice until Exit is selected.
8. Stop.

Program:

```
/* PART B - PROGRAM 3
   Circular Queue (floating point data) using array
   Operations: Insert, Delete, Display, Overflow/Underflow demonstration
*/
#include <stdio.h>
#define MAX 5

float cq[MAX];
int front = -1, rear = -1;

void insert(float val)
{
    if ((rear + 1) % MAX == front)
    {
        printf("Circular Queue Overflow! Cannot insert %.2f\n", val);
        return;
    }
    if (front == -1)
        front = 0;
    rear = (rear + 1) % MAX;
    cq[rear] = val;
    printf("%.2f inserted into circular queue.\n", val);
}

void deleteElem()
{
    if (front == -1)
    {
        printf("Circular Queue Underflow! Nothing to delete.\n");
        return;
    }
```

```
    printf("Deleted element: %.2f\n", cq[front]);
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display()
{
    int i;
    if (front == -1)
    {
        printf("Circular Queue is empty.\n");
        return;
    }
    printf("Circular Queue contents: ");
    i = front;
    while (1)
    {
        printf("%.2f ", cq[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

void demoOverflowUnderflow()
{
    int i;
    float v = 1.5;
    front = rear = -1;
    printf("Demonstrating Overflow: inserting %d elements into circular queue of size %d\n", MAX + 2,
MAX);
    for (i = 1; i <= MAX + 2; i++)
    {
        insert(v);
        v += 1.5;
    }
    printf("\nDemonstrating Underflow: deleting %d elements\n", MAX + 2);
    for (i = 1; i <= MAX + 2; i++)
        deleteElem();
}

int main()
{
    int choice;
    float val;
    do
    {
        printf("\n----- CIRCULAR QUEUE (Floating Point Data) -----\n");
        printf("1. Insert an element\n");
        printf("2. Delete an element\n");
        printf("3. Display contents\n");
        printf("4. Demonstrate Overflow and Underflow\n");
```

```
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value to insert: ");
                scanf("%f", &val);
                insert(val);
                break;
            case 2: deleteElem(); break;
            case 3: display(); break;
            case 4: demoOverflowUnderflow(); break;
            case 5: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 5);
    return 0;
}
```

Sample Output:

```
----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Enter value to insert: 10.00 inserted into circular queue.

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Enter value to insert: 20.00 inserted into circular queue.

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Enter value to insert: 30.00 inserted into circular queue.

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
```

```
Enter your choice: Circular Queue contents: 10.00 20.00 30.00

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Deleted element: 10.00

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Circular Queue contents: 20.00 30.00

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Demonstrating Overflow: inserting 7 elements into circular queue of size 5
1.50 inserted into circular queue.
3.00 inserted into circular queue.
4.50 inserted into circular queue.
6.00 inserted into circular queue.
7.50 inserted into circular queue.
Circular Queue Overflow! Cannot insert 9.00
Circular Queue Overflow! Cannot insert 10.50

Demonstrating Underflow: deleting 7 elements
Deleted element: 1.50
Deleted element: 3.00
Deleted element: 4.50
Deleted element: 6.00
Deleted element: 7.50
Circular Queue Underflow! Nothing to delete.
Circular Queue Underflow! Nothing to delete.

----- CIRCULAR QUEUE (Floating Point Data) -----
1. Insert an element
2. Delete an element
3. Display contents
4. Demonstrate Overflow and Underflow
5. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- Why is a circular queue preferred over a simple linear queue?
- How is the "queue full" condition detected in a circular queue?
- Why does a circular array of size MAX effectively hold only MAX-1 elements in this scheme?

- What is the role of the modulo (%) operator in circular queue operations?
- Compare a circular queue with a circular linked list implementation of a queue.

Program B-4 Doubly Linked List of Employee Data — used as a Double Ended Queue

Course Outcomes Mapped: CO1, CO2

Aim:

To implement a Doubly Linked List (DLL) of Employee records (SSN, Name, Dept, Designation, Sal, PhNo) built using end insertion, supporting insertion/deletion at both the front and the end, and to demonstrate how the DLL can function as a Double Ended Queue (Deque).

Algorithm:

1. Start.
2. Define a structure Employee with data members SSN, Name, Dept, Designation, Sal, PhNo and pointers prev and next.
3. createDLL(): for each of the N employees, allocate a new node, read its data and insert it at the end of the list (end insertion), updating head/tail and prev/next pointers.
4. displayAndCount(): traverse from head to tail printing every node's data and counting the number of nodes.
5. insertEnd()/insertFront(): allocate a new node and link it after tail (updating tail) or before head (updating head), adjusting prev and next pointers on both sides.
6. deleteEnd()/deleteFront(): unlink the node at tail or head, update tail/head and the neighbouring prev/next pointer, and free the removed node.
7. demoDeque(): insert one employee at the rear end and one at the front end, display the list, then delete one employee from the front and one from the rear, demonstrating that both ends of the DLL support insertion and deletion just like a Double Ended Queue.
8. Repeat according to the user's menu choice until Exit is selected.
9. Stop.

Program:

```
/* PART B - PROGRAM 4
   Doubly Linked List (DLL) of Employee Data: SSN, Name, Dept, Designation, Sal, PhNo
   a. Create DLL of N employees using end insertion
   b. Display status and count nodes
   c. Insertion and Deletion at End
   d. Insertion and Deletion at Front
   e. Demonstrate DLL used as a Double Ended Queue
*/
#include <stdio.h>
#include <stdlib.h>

struct Employee
{
    char SSN[15];
    char Name[30];
    char Dept[20];
    char Designation[20];
    float Sal;
    char PhNo[15];
    struct Employee *prev, *next;
};

struct Employee *head = NULL, *tail = NULL;
```

```
struct Employee *readEmployee()
{
   struct Employee *e = (struct Employee *) malloc(sizeof(struct Employee));
   printf("Enter SSN: ");
   scanf("%s", e->SSN);
   printf("Enter Name: ");
   scanf("%s", e->Name);
   printf("Enter Dept: ");
   scanf("%s", e->Dept);
   printf("Enter Designation: ");
   scanf("%s", e->Designation);
   printf("Enter Salary: ");
   scanf("%f", &e->Sal);
   printf("Enter Phone Number: ");
   scanf("%s", e->PhNo);
   e->prev = e->next = NULL;
   return e;
}

void insertEnd(struct Employee *e)
{
   if (head == NULL)
   {
      head = tail = e;
      return;
   }
   tail->next = e;
   e->prev = tail;
   tail = e;
}

void insertFront(struct Employee *e)
{
   if (head == NULL)
   {
      head = tail = e;
      return;
   }
   e->next = head;
   head->prev = e;
   head = e;
}

void createDLL()
{
   int n, i;
   printf("Enter number of employees: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      printf("\n--- Employee %d ---\n", i + 1);
      struct Employee *e = readEmployee();
      insertEnd(e);
   }
```

```
    printf("\nDoubly Linked List created using end insertion.\n");
}

void displayAndCount()
{
    struct Employee *p = head;
    int count = 0;
    if (head == NULL)
    {
        printf("DLL is empty.\n");
        return;
    }
    printf("\n%-10s%-12s%-10s%-15s%-10s%-12s\n", "SSN", "Name", "Dept", "Designation", "Salary",
"PhNo");
    printf("-------------------------------------------------------------------------\n");
    while (p != NULL)
    {
        printf("%-10s%-12s%-10s%-15s%-10.2f%-12s\n", p->SSN, p->Name, p->Dept,
              p->Designation, p->Sal, p->PhNo);
        count++;
        p = p->next;
    }
    printf("Total number of nodes: %d\n", count);
}

void deleteEnd()
{
    if (tail == NULL)
    {
        printf("DLL is empty. Nothing to delete.\n");
        return;
    }
    printf("Deleted employee with SSN: %s\n", tail->SSN);
    if (head == tail)
    {
        free(tail);
        head = tail = NULL;
        return;
    }
    struct Employee *temp = tail;
    tail = tail->prev;
    tail->next = NULL;
    free(temp);
}

void deleteFront()
{
    if (head == NULL)
    {
        printf("DLL is empty. Nothing to delete.\n");
        return;
    }
    printf("Deleted employee with SSN: %s\n", head->SSN);
    if (head == tail)
```

```
    {
        free(head);
        head = tail = NULL;
        return;
    }
    struct Employee *temp = head;
    head = head->next;
    head->prev = NULL;
    free(temp);
}

/* Demonstrates DLL as a Double Ended Queue: insert/delete allowed at both ends */
void demoDeque()
{
    printf("\n--- Demonstrating DLL as a Double Ended Queue (Deque) ---\n");
    printf("Inserting a new employee at REAR end (like enqueue-rear):\n");
    struct Employee *e1 = readEmployee();
    insertEnd(e1);

    printf("\nInserting a new employee at FRONT end (like enqueue-front):\n");
    struct Employee *e2 = readEmployee();
    insertFront(e2);

    printf("\nCurrent DLL after insertions from both ends:\n");
    displayAndCount();

    printf("\nDeleting an employee from FRONT end (like dequeue-front):\n");
    deleteFront();

    printf("\nDeleting an employee from REAR end (like dequeue-rear):\n");
    deleteEnd();

    printf("\nDLL after deletions from both ends:\n");
    displayAndCount();
}

int main()
{
    int choice, sub;
    do
    {
        printf("\n----- DOUBLY LINKED LIST OF EMPLOYEE DATA -----\n");
        printf("1. Create DLL using end insertion\n");
        printf("2. Display status and count of nodes\n");
        printf("3. Insert/Delete at End\n");
        printf("4. Insert/Delete at Front\n");
        printf("5. Demonstrate DLL as Double Ended Queue\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: createDLL(); break;
```

```
            case 2: displayAndCount(); break;
            case 3:
                printf("1.Insert at End 2.Delete at End\nChoice: ");
                scanf("%d", &sub);
                if (sub == 1) { struct Employee *e = readEmployee(); insertEnd(e); }
                else if (sub == 2) deleteEnd();
                else printf("Invalid sub-choice.\n");
                break;
            case 4:
                printf("1.Insert at Front 2.Delete at Front\nChoice: ");
                scanf("%d", &sub);
                if (sub == 1) { struct Employee *e = readEmployee(); insertFront(e); }
                else if (sub == 2) deleteFront();
                else printf("Invalid sub-choice.\n");
                break;
            case 5: demoDeque(); break;
            case 6: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 6);
    return 0;
}
```

Sample Output:

```
----- DOUBLY LINKED LIST OF EMPLOYEE DATA -----
1. Create DLL using end insertion
2. Display status and count of nodes
3. Insert/Delete at End
4. Insert/Delete at Front
5. Demonstrate DLL as Double Ended Queue
6. Exit
Enter your choice: Enter number of employees:
--- Employee 1 ---
Enter SSN: Enter Name: Enter Dept: Enter Designation: Enter Salary: Enter Phone Number:
--- Employee 2 ---
Enter SSN: Enter Name: Enter Dept: Enter Designation: Enter Salary: Enter Phone Number:
Doubly Linked List created using end insertion.

----- DOUBLY LINKED LIST OF EMPLOYEE DATA -----
1. Create DLL using end insertion
2. Display status and count of nodes
3. Insert/Delete at End
4. Insert/Delete at Front
5. Demonstrate DLL as Double Ended Queue
6. Exit
Enter your choice:
SSN      Name      Dept     Designation    Salary    PhNo
---------------------------------------------------------------------------
E001     Anitha    CSE      Engineer       55000.00  9876543210
E002     Ravi      ISE      Manager        75000.00  9876500000
Total number of nodes: 2
```

```
----- DOUBLY LINKED LIST OF EMPLOYEE DATA -----
1. Create DLL using end insertion
2. Display status and count of nodes
3. Insert/Delete at End
4. Insert/Delete at Front
5. Demonstrate DLL as Double Ended Queue
6. Exit
Enter your choice:
--- Demonstrating DLL as a Double Ended Queue (Deque) ---
Inserting a new employee at REAR end (like enqueue-rear):
Enter SSN: Enter Name: Enter Dept: Enter Designation: Enter Salary: Enter Phone Number:
Inserting a new employee at FRONT end (like enqueue-front):
Enter SSN: Enter Name: Enter Dept: Enter Designation: Enter Salary: Enter Phone Number:
Current DLL after insertions from both ends:

SSN      Name       Dept     Designation    Salary    PhNo
---------------------------------------------------------------------------
E004     Suresh     MECH     Technician     40000.00  9123456780
E001     Anitha     CSE      Engineer       55000.00  9876543210
E002     Ravi       ISE      Manager        75000.00  9876500000
E003     Kavya      ECE      Analyst        50000.00  9998887770
Total number of nodes: 4

Deleting an employee from FRONT end (like dequeue-front):
Deleted employee with SSN: E004

Deleting an employee from REAR end (like dequeue-rear):
Deleted employee with SSN: E003

DLL after deletions from both ends:

SSN      Name       Dept     Designation    Salary    PhNo
---------------------------------------------------------------------------
E001     Anitha     CSE      Engineer       55000.00  9876543210
E002     Ravi       ISE      Manager        75000.00  9876500000
Total number of nodes: 2

----- DOUBLY LINKED LIST OF EMPLOYEE DATA -----
1. Create DLL using end insertion
2. Display status and count of nodes
3. Insert/Delete at End
4. Insert/Delete at Front
5. Demonstrate DLL as Double Ended Queue
6. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- How does a doubly linked list differ from a singly linked list?
- What is a Double Ended Queue (Deque)? How does it differ from a normal queue?
- Why can a DLL be used to implement a Deque efficiently, unlike a singly linked list?
- What is the space overhead of a DLL node compared to a singly linked list node?
- What are the time complexities of insertion and deletion at both ends of a DLL?

Program B-5 Binary Search Tree (BST) — Creation, Traversal and Search

Course Outcomes Mapped: CO1, CO2

Aim:

To implement a Binary Search Tree (BST) of N integers, traverse it using Inorder, Preorder and Postorder traversals, and search the BST for a given KEY, reporting an appropriate message.

Algorithm:

1. Start.
2. Define a structure Node with members data, left and right.
3. insert(root, val): if root is NULL, create and return a new node with value val; if val < root->data, recursively insert into the left subtree; if val > root->data, recursively insert into the right subtree; return root.
4. createBST(): read N integers one at a time and insert each of them into the BST using insert().
5. inorder/preorder/postorder: perform the standard recursive traversals of the BST (as in Program 5, Part A) and display the resulting sequences.
6. search(key): starting at the root, compare key with the current node's data; if equal, report FOUND; if key is smaller, move to the left child; if key is larger, move to the right child; repeat until the key is found or a NULL pointer is reached (report NOT FOUND).
7. Repeat according to the user's menu choice until Exit is selected.
8. Stop.

Program:

```
/* PART B - PROGRAM 5
   Binary Search Tree (BST) of Integers
   a. Create a BST of N integers
   b. Traverse the BST in Inorder, Preorder and Postorder
   c. Search the BST for a given element (KEY)
*/
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *left, *right;
};

struct Node *root = NULL;

struct Node *createNode(int val)
{
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *t, int val)
{
    if (t == NULL)
        return createNode(val);
```

```
    if (val < t->data)
        t->left = insert(t->left, val);
    else if (val > t->data)
        t->right = insert(t->right, val);
    /* duplicate values are ignored */
    return t;
}

void createBST()
{
    int n, i, val;
    root = NULL;
    printf("Enter number of integers (N) to insert into BST: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &val);
        root = insert(root, val);
    }
    printf("Binary Search Tree created successfully.\n");
}

void inorder(struct Node *t)
{
    if (t != NULL)
    {
        inorder(t->left);
        printf("%d ", t->data);
        inorder(t->right);
    }
}

void preorder(struct Node *t)
{
    if (t != NULL)
    {
        printf("%d ", t->data);
        preorder(t->left);
        preorder(t->right);
    }
}

void postorder(struct Node *t)
{
    if (t != NULL)
    {
        postorder(t->left);
        postorder(t->right);
        printf("%d ", t->data);
    }
}

void searchBST()
```

```
{
   int key;
   struct Node *p = root;
   printf("Enter KEY to search: ");
   scanf("%d", &key);
   while (p != NULL)
   {
      if (key == p->data)
      {
         printf("Element %d FOUND in the BST.\n", key);
         return;
      }
      else if (key < p->data)
         p = p->left;
      else
         p = p->right;
   }
   printf("Element %d NOT FOUND in the BST.\n", key);
}

int main()
{
   int choice;
   do
   {
      printf("\n----- BINARY SEARCH TREE OF INTEGERS -----\n");
      printf("1. Create a BST of N Integers\n");
      printf("2. Traverse the BST (Inorder, Preorder, Postorder)\n");
      printf("3. Search the BST for a given element (KEY)\n");
      printf("4. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1: createBST(); break;
         case 2:
            if (root == NULL) { printf("BST is empty!\n"); break; }
            printf("Inorder   : "); inorder(root); printf("\n");
            printf("Preorder  : "); preorder(root); printf("\n");
            printf("Postorder : "); postorder(root); printf("\n");
            break;
         case 3:
            if (root == NULL) { printf("BST is empty!\n"); break; }
            searchBST();
            break;
         case 4: printf("Exiting...\n"); break;
         default: printf("Invalid choice!\n");
      }
   } while (choice != 4);
   return 0;
}
```

Sample Output:

----- BINARY SEARCH TREE OF INTEGERS -----
1. Create a BST of N Integers
2. Traverse the BST (Inorder, Preorder, Postorder)
3. Search the BST for a given element (KEY)
4. Exit
Enter your choice: Enter number of integers (N) to insert into BST: Enter element 1: Enter element 2: Enter element 3: Enter element 4: Enter element 5: Enter element 6: Enter element 7: Binary Search Tree created successfully.

----- BINARY SEARCH TREE OF INTEGERS -----
1. Create a BST of N Integers
2. Traverse the BST (Inorder, Preorder, Postorder)
3. Search the BST for a given element (KEY)
4. Exit
Enter your choice: Inorder : 20 30 40 50 60 70 80
Preorder : 50 30 20 40 70 60 80
Postorder : 20 40 30 60 80 70 50

----- BINARY SEARCH TREE OF INTEGERS -----
1. Create a BST of N Integers
2. Traverse the BST (Inorder, Preorder, Postorder)
3. Search the BST for a given element (KEY)
4. Exit
Enter your choice: Enter KEY to search: Element 40 FOUND in the BST.

----- BINARY SEARCH TREE OF INTEGERS -----
1. Create a BST of N Integers
2. Traverse the BST (Inorder, Preorder, Postorder)
3. Search the BST for a given element (KEY)
4. Exit
Enter your choice: Enter KEY to search: Element 90 NOT FOUND in the BST.

----- BINARY SEARCH TREE OF INTEGERS -----
1. Create a BST of N Integers
2. Traverse the BST (Inorder, Preorder, Postorder)
3. Search the BST for a given element (KEY)
4. Exit
Enter your choice: Exiting...

Viva-Voce Questions:

- Define a Binary Search Tree. What property must every node satisfy?
- What is the time complexity of search, insertion and deletion in a BST in the average and worst case?
- Which traversal of a BST always produces the elements in sorted order? Why?
- How would you delete a node with two children from a BST?
- What is a degenerate (skewed) BST, and why does it lead to O(n) search time?

Program B-6 Employee Record Management System using Hashing (Linear Probing)

Course Outcomes Mapped: CO1, CO2

Aim:

To develop an Employee Record Management System using Hashing, where records are stored using Employee ID as the key with hash function H(K) = K mod m, collisions are resolved using Linear Probing, and to support search and display of the hash table along with memory locations.

Algorithm:

1. Start.
2. Declare a hash table of size m (an array of Employee records) and initialise every slot as empty.
3. hashFunction(key): return key mod m as the home address of the key.
4. insert(id, name): compute index = hashFunction(id); while the slot at index is occupied, move to the next slot using index = (index+1) mod m (linear probing); insert the record into the first free slot found.
5. search(id): compute index = hashFunction(id); examine slots starting at index; if the slot holds the given id, report FOUND along with its location; else move to the next slot using linear probing; if all m slots have been examined without a match, report NOT FOUND.
6. display(): print every location of the hash table from 0 to m-1, along with the Employee ID and Name stored there, or "empty" if the slot is unoccupied.
7. Repeat according to the user's menu choice until Exit is selected.
8. Stop.

Program:

```
/* PART B - PROGRAM 6
   Employee Record Management System using Hashing
   Hash function: H(K) = K mod m,  Collision resolution: Linear Probing
   Operations: Create hash table, Insert, Search, Display with memory locations
*/
#include <stdio.h>
#include <string.h>
#define M 10   /* table size (m) */

struct Employee
{
    int empID;
    char name[30];
    int isOccupied;   /* 0 = empty slot, 1 = occupied */
};

struct Employee hashTable[M];

void initTable()
{
    int i;
    for (i = 0; i < M; i++)
        hashTable[i].isOccupied = 0;
}

int hashFunction(int key)
{
    return key % M;
}
```

```
void insertRecord(int id, char name[])
{
   int index = hashFunction(id);
   int start = index, count = 0;

   while (hashTable[index].isOccupied == 1 && count < M)
   {
      index = (index + 1) % M;   /* linear probing */
      count++;
   }

   if (count == M)
   {
      printf("Hash Table is FULL. Cannot insert Employee ID %d.\n", id);
      return;
   }

   hashTable[index].empID = id;
   strcpy(hashTable[index].name, name);
   hashTable[index].isOccupied = 1;

   if (index == start)
      printf("Employee ID %d inserted at location %d (no collision).\n", id, index);
   else
      printf("Employee ID %d inserted at location %d (collision resolved by linear probing from %d).\n",
            id, index, start);
}

void searchRecord(int id)
{
   int index = hashFunction(id);
   int count = 0;

   while (count < M)
   {
      if (hashTable[index].isOccupied == 1 && hashTable[index].empID == id)
      {
         printf("Employee ID %d FOUND at location %d -> Name: %s\n",
               id, index, hashTable[index].name);
         return;
      }
      index = (index + 1) % M;
      count++;
   }
   printf("Employee ID %d NOT FOUND in the hash table.\n", id);
}

void displayTable()
{
   int i;
   printf("\n%-12s%-12s%-20s\n", "Location", "EmpID", "Name");
   printf("-----------------------------------------\n");
   for (i = 0; i < M; i++)
```

```
    {
        if (hashTable[i].isOccupied)
            printf("%-12d%-12d%-20s\n", i, hashTable[i].empID, hashTable[i].name);
        else
            printf("%-12d%-12s%-20s\n", i, "--", "empty");
    }
}

int main()
{
    int choice, id;
    char name[30];
    initTable();

    do
    {
        printf("\n----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----\n");
        printf("(Table size m = %d, H(K) = K mod %d, Linear Probing)\n", M, M);
        printf("1. Insert an Employee record\n");
        printf("2. Search an Employee record using Employee ID\n");
        printf("3. Display the Hash Table with memory locations\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter Employee ID: ");
                scanf("%d", &id);
                printf("Enter Employee Name: ");
                scanf("%s", name);
                insertRecord(id, name);
                break;
            case 2:
                printf("Enter Employee ID to search: ");
                scanf("%d", &id);
                searchRecord(id);
                break;
            case 3: displayTable(); break;
            case 4: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

Sample Output:

```
----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
```

```
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID: Enter Employee Name: Employee ID 101 inserted at location 1
(no collision).

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID: Enter Employee Name: Employee ID 111 inserted at location 2
(collision resolved by linear probing from 1).

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID: Enter Employee Name: Employee ID 121 inserted at location 3
(collision resolved by linear probing from 1).

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID: Enter Employee Name: Employee ID 104 inserted at location 4
(no collision).

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice:
Location    EmpID       Name
---------------------------------------------
0           --          empty
1           101           Anil
2           111           Bala
3           121           Chetan
4           104           Deepa
5           --          empty
6           --          empty
7           --          empty
8           --          empty
9           --          empty

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
```

```
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID to search: Employee ID 111 FOUND at location 2 -> Name: Bala

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Enter Employee ID to search: Employee ID 999 NOT FOUND in the hash table.

----- EMPLOYEE RECORD MANAGEMENT USING HASHING -----
(Table size m = 10, H(K) = K mod 10, Linear Probing)
1. Insert an Employee record
2. Search an Employee record using Employee ID
3. Display the Hash Table with memory locations
4. Exit
Enter your choice: Exiting...
```

Viva-Voce Questions:

- What is hashing? Define a hash function and a collision.
- What is Linear Probing? What is primary clustering, and how does it affect Linear Probing?
- Compare Linear Probing with other collision resolution techniques (Quadratic Probing, Chaining).
- What is the load factor of a hash table, and how does it affect performance?
- What is the average time complexity of search, insert and delete in a well-designed hash table?

References

- Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, “Fundamentals of Data Structures in C”, 2nd Edition, Universities Press, 2023.
- Reema Thareja, “Data Structures using C”, 3rd Edition, Oxford University Press, 2023.
- A. M. Tenenbaum, Y. Langsam and M. J. Augenstein, “Data Structures using C”, Pearson/PHI, 1989.
- Seymour Lipschutz, “Data Structures”, Schaum's Outline Series, Revised 1st Edition, McGraw-Hill, 2014.
- Herbert Schildt, “C: The Complete Reference”, 4th Edition, McGraw-Hill, 2000.
- VTU 2025 Scheme Syllabus — Data Structures Laboratory (Course Code: 1BCSL306), Visvesvaraya Technological University, Belagavi.
- Dev-C++ (MinGW-w64 GCC toolchain) — https://www.bloodshed.net/devcpp.html

******* END OF MANUAL *******